# Object-oriented analysis, design and programming OAP200V

Course overview - Fall 2026

Katerina Mangaroska, Boban Vesin

September 6, 2026

# Contents


**1 Introduction to the course** **3**

**2 Course schedule** **10**

**3 The course topics - Analysis and Design** **11**
3.1 Introduction to the course . . . . . 11
3.2 Introduction to System Development . . . . . 11
3.3 Introduction to Object-Oriented Paradigm . . . . . 12
3.4 System Analysis Activities . . . . . 12
3.5 System Analysis Activities . . . . . 12
3.6 System Design Activities . . . . . 12
3.7 System Design Activities . . . . . 13
3.8 Designing the User-Interface and Deploying the New System 13

**4 The course topics - Programming** **14**
4.1 Introduction to the course . . . . . 15
4.2 Eclipse IDE . . . . . 15
4.3 Advanced Array Concepts . . . . . 16
4.4 Advanced Inheritance Concepts . . . . . 16
4.5 Revision control . . . . . 18
4.6 Software project management with Maven . . . . . 18
4.7 Exception Handling . . . . . 19
4.8 File Input and Output . . . . . 20
4.9 Database access . . . . . 21
4.10 User Interface . . . . . 22
4.11 Threads and multitasking . . . . . 23
4.12 Final exam guidelines . . . . . 24

**5 Mandatory Assignment** **25**

**6 The running example** **26**
6.1 Problem Description: Cinema Manager . . . . . 26
6.2 Requirements Analysis . . . . . 27
6.3 Structure of the Code . . . . . 29

**7 The exam** **33**
7.1 Your task . . . 33
7.1.1 Part 1: Problem Analysis and System Design . . . 33
7.1.2 Part 2: Implementation of the app . . . 35
7.1.3 Part 3. Final presentation demo . . . 36
7.2 Java app requirements . . . 37
7.2.1 Minimum requirements (for minimum passing grade) . 37
7.2.2 Optional requirements (for higher grade) . . . 37
7.2.3 Specific requirements . . . 38
7.3 Evaluation . . . 38
7.3.1 Submission . . . 39
7.4 Grading Criteria . . . 39
7.5 Important notes . . . 40
7.6 Problem Analysis and System Design Report . . . 41
7.6.1 Introduction . . . 41
7.6.2 Problem Analysis . . . 41
7.6.3 System Design . . . 42
7.6.4 Database Design . . . 42
7.6.5 User Interface Design . . . 42
7.6.6 Implementation Overview . . . 42
7.6.7 Testing Insights . . . 43
7.6.8 Challenges and Limitations . . . 43
7.6.9 Individual Contributions . . . 43
7.6.10 Conclusion . . . 43

**8 Project description for the exam** **44**
8.1 Problem: Private Book Library Manager . . . 44
8.2 Suggested Database . . . 45

**9 Uses of Java Programming** **47**

**10 Software** **49**

**11 Detailed guidelines for setup** **52**
11.1 Setting Up JavaFX + Maven Project in Eclipse . . . 52
11.1.1 1. Windows Setup . . . 52
11.1.2 2. macOS Setup . . . 55

# Preface

This document provides all the necessary information about the *Object-Oriented Analysis, Design, and Programming* course for the Fall Semester 2026 at the *University of South-Eastern Norway*, campus Vestfold. The document's structure follows the course structure, aligning with the organization of the modules in Canvas.

## Content of the document

The document contains the following chapters:

- **Introduction to the course.** This chapter provides all the necessary technical details about the execution of the course: timetable, course content, and structure, an overview of learning activities, assignments, deadlines, mandatory and optional literature, evaluation, and grading (chapter 1).
- **Course schedule**. This chapter presents the detailed schedule of the course, with topics and activities (chapter 2).
- **The course topics**. chapter 3 and chapter 4 present the overview of topics that will be taught during the course and mandatory and optional learning material that covers those topics.
- **Mandatory assignment** (chapter 5). The guidelines and requirements for the delivery of a mandatory assignment are presented in this chapter.
- **Running example**. This section contains the details of the running example used during lectures (chapter 6).
- **The exam**. This section presents the details of the exam (chapter 7).
- **Project description for the exam**. This section contains the project description that students will work on (in teams) for the final exam (chapter 8).

- **Uses of Java programming**. Description of the practical implications of the course (chapter 9).
- **Software**. Details on the software that will be used during the course (chapter 10).

# 1

# Introduction to the course

## Basic information

| | |
|---|---|
| Name of the course: | **Object-oriented analysis, design and programming** |
| Norwegian name: | Objektorientert analyse, design og programmering |
| Code of the course: | OAP200V |
| Credits: | 15 |
| Time-slot: | Monday, Wednesday lectures: 10:15-13:00, labs 13:15-14:00 |
| Room (lectures): | See schedule in Canvas |
| Room (labs): | See schedule in Canvas |
| Lecturers: | Katerina Mangaroska, Boban Vesin |

## Timeline

| | |
|---|---|
| Start of the course | 17.8.2026 |
| Delivery of the mandatory assignment | 11.11.2026, 23:59 |
| Presentation of the mandatory assignment | 9.11.2026 and 11.11.2026 |
| End of the semester | 16.11.2026 |
| Delivery of the final project and app | Exam date (TBD) |
| Final demo (Oral exam) | Oral exam date (TBD) |
| Feedback on the final report | Three weeks after |

## Academic content on the subject

The course introduces graphical modeling language and techniques for object-oriented analysis, design, and documentation of information systems. It will provide skills in seeing the connection between models and program code and understanding systems through model descriptions.

The students will further develop their knowledge of object-oriented programming by creating larger applications.

This course will provide hands-on experience in practical software design and programming of desktop applications. It is structured in two parts: students will work in parallel on the app's design and implementation.

## Prerequisites

The students should have general Java programming skills corresponding to the Object-oriented programming course.

## Outcomes

**Knowledge** - After completing the course, the students should have knowledge of:

- how UML models can be used to show and understand the system from different perspectives
- how UML models can be used for the analysis of user requirements and software design
- modeling techniques in UML
- the connection between UML syntax and specific programming code
- persistence: local file storage, serialization and database access
- streams, network classes and I/O
- threads and multitasking
- user interface design
- polymorphism and overloading
- anonymous and inner classes
- non-linear data structures

**Skills** - After completing the course the student will have the following skills:

- create structure diagrams for simple and more complex systems
- analyze existing software
- model new software

- visualize and plan software development process
- use different classes for persistence
- use textual and binary serialization
- Program threads and perform multitasking
- Program with non-linear data structures

**General competence** - After completing the course, the students should be able to:

- choose good perspectives for studying program systems
- analyse relevant IT professional issues
- have good expertise in object-oriented subjects
- build larger object-oriented user programs
- understand diagrams of larger structures and program design
- reflect on own professional practice and adjust this under the guidance

## Learning activities

Lectures and problem solving. In the lectures, the basis for being able to develop skills in programming is reviewed. They are also the basis for being able to solve practice tasks in the subject. Throughout the course, the students have the following tasks:

- Theoretical lectures.
- Practical lectures.
- Mandatory assignment (submission and oral presentation).
- Delivery of the final project and the app.
- The final demo.

### Team Work

Students will carry out the activities in teams of five to six members. Students will be able to choose their own teams.

## Lecturers and examiners

**Katerina Mangaroska** (personal page) and **Boban Vesin** (personal page) are responsible for the course. Their office is at Campus Vestfold, Room 136, Block C3. Recommended means of communication will be via email:

- katerina.mangaroska@usn.no
- boban.vesin@usn.no

For questions and other inquiries, you can (i) email the lecturer or (ii) make an appointment for a Zoom meeting. Please use regular email (instead of Canvas messages), especially when the entire team and both instructors are included in the conversation.

## Forms of Assessments

The final grade in this course is based on two main components: a written submission and an oral examination. Both parts are conducted in teams and together assess students' knowledge and application of object-oriented analysis, design, and programming principles.

### 1. Written Exam (60%)

The written exam is a project-based home exam carried out in teams over the duration of the semester. It consists of:

- **A written report**, submitted via Canvas, documenting the problem analysis, system design, implementation details and testing efforts.
- **The developed application**, which includes the full source code and must reflect the design outlined in the report.

The report must clearly describe each team member's individual contributions. This part of the exam constitutes 60% of the final grade.

### 2. Oral Exam (40%)

The oral exam is a live, team-based presentation and demonstration of the developed application. It includes:

- A presentation of the design choices and architecture.
- A demonstration of the application's features and UI.
- A Q&A session where each team member must demonstrate their understanding and contributions.

The oral exam constitutes 40% of the final grade.

### Mandatory Assignment (Work Requirement)

To qualify for the final exam, each team must complete and present one **mandatory assignment** which focuses on the initial analysis and design of the application. This work requirement must be approved before the team is allowed to proceed to the exam phase.

Details about the mandatory assignment are provided in chapter 5. More information about the final exam is presented in chapter 7.

## Evaluation Criteria

Both the written and oral parts of the exam are evaluated independently and each can yield a maximum of **100 points**.

### Written Exam Evaluation (100 points total)

The written exam includes the submission of a comprehensive report and a working application. The evaluation is based on the following criteria:

| **Evaluation Criteria** | **Points** |
|---|---|
| Content and structure of the report | 15 |
| Software design | 25 |
| UX and UI design | 20 |
| Features and complexity | 40 |
| **Total** | **100** |

### Oral Exam Evaluation (100 points total)

The oral exam includes a group presentation, demonstration of the developed application, and a Q&A session with the examiners. The evaluation is based on the following criteria:

| **Evaluation Criteria** | **Points** |
|---|---|
| Presentation structure and clarity | 25 |
| Depth of problem analysis and design explanation | 25 |
| Demonstration of application features | 20 |
| Team communication and collaboration | 15 |
| Individual contribution and question handling | 15 |
| **Total** | **100** |

The final grade is automatically calculated in WISEflow using the weighted average formula. Each student is assessed individually, particularly during the oral exam and based on the documentation of individual contributions in the written report.

## Grading scale(for both exams)

A detailed explanation of the grading is presented in chapter 3. The grading scale is in the range of A to F, where A is the best grade, and F is failed. Grade scale from A to F:

| **Overall points** | **Grade** |
|---|---|
| 91-100 | A |
| 81-90 | B |
| 71-80 | C |
| 61-70 | D |
| 51-60 | E |
| <51 | F |

## Evaluation of the course

The course will be evaluated regularly to maintain high quality. Students are expected to participate actively in evaluations, surveys, meetings and more to help raise the education quality of the study program. The evaluations are in line with USN's quality system.

## Literature

Required:

- **Introduction to systems analysis and design: an agile, iterative approach**
  Satzinger, John Jackson, Robert Burd, Stephen D.
  Cengage learning, 2014
- Joyce Farrell
  **Java Programming, Ninth Edition**
  Course Technology | Cengage Learning, 2022, ISBN-13: 978-1-337-39707-0.
- Material from *Canvas*

Optional:

(i) Holmstedt, Viggo
**Objektorientert programmering med Java : effektiv problemløsning i systemutvikling**
Fagbokforl, 2002, ISBN: 8276747485;

(ii) additional literature is presented in the Canvas (section Pensum/reading list)

# Required software

## Programming

- JRE, JDK: `https://www.oracle.com/java/technologies/javase-jre8-downloads.html`
- Eclipse IDE: `https://www.eclipse.org/ide/`

The details about the used software and installation process are presented in chapter 10.

## UML design:

- Microsoft Visio: `https://apps.usn.no/login`
- Online editor: `https://www.draw.io/`
- Miro: `https://miro.com/diagramming/uml-diagram/`

# 2

# Course schedule

The official and up-to-date schedule will be presented at the Canvas home page at all times. The course schedule can be subject to changes during the semester, so please pay attention to the announcements made in Canvas.

# 3

# The course topics - Analysis and Design

This chapter presents the topics covered in the analysis and design section of the course.

## 3.1 Introduction to the course

**Topic:** Introduction to the course

- Go through the course syllabus together with the students;
- Explain the exam, the assessment, and the grading process;
- Discuss expectations and students' personal goals (send a Google survey);
- Allow for questions and suggestions.

## 3.2 Introduction to System Development

**Lecture:** Introduction to System Development

1. System development life-cycle
2. Approaches to system development
3. Project planning and project management

**Readings:** Systems Analysis and Design in a Changing World (Chapter 1, pages 4-9; Chapter 10, pages 296-318; Chapter 11, pages 326-356).

## 3.3 Introduction to Object-Oriented Paradigm

**Lecture:** Introduction to Object-Oriented Paradigm

1. What is object-oriented paradigm?
2. A PIE: Abstraction, Polymorphism, Inheritance, and Encapsulation
3. Introduction to UML

**Readings:** Objektorientert analyse og design med UML (kapittel 1, pages 2-15; kapittel 2, pages 17-25).

## 3.4 System Analysis Activities

**Lecture:** Requirements Analysis

1. Investigating system requirements
2. Identifying user stories and use cases

**Readings:** Systems Analysis and Design in a Changing World (Chapter 2, pages 38-63; Chapter 3, pages 71-87).

## 3.5 System Analysis Activities

**Lecture:** Fundamental Principles of Object-Oriented Analysis

1. Domain modeling – domain model class diagram
2. Use case modeling – use case descriptions and use case diagram

**Readings:** Systems Analysis and Design in a Changing World (Chapter 4, Chapter 5), Objektorientert analyse og design med UML (kapittel 3, pages 29-40, kapittel 6, pages 66-72)

## 3.6 System Design Activities

**Lecture:** Fundamental Principles of Object-Oriented Design

1. Design Classes and the Design Class Diagram
2. Class modeling design techniques

**Readings:** Systems Analysis and Design in a Changing World (Chapter 12), Objektorientert analyse og design med UML (kapittel 4, kapittel 9)

## 3.7 System Design Activities

**Lecture:** Fundamental Principles of Object-Oriented Design

1. Use-case realization with sequence diagrams
2. From object design to Java code

**Readings:** Systems Analysis and Design in a Changing World (Chapter 13), Objektorientert analyse og design med UML (kapittel 7)

## 3.8 Designing the User-Interface and Deploying the New System

**Lecture:** Designing the User-Interface and Deploying the New System

1. Designing guidelines
2. Human-computer interaction (HCI)
3. Deploying the new system

**Readings:** Systems Analysis and Design in a Changing World (Chapter 14), Objektorientert analyse og design med UML (kapittel 8, pages 82-87)

# 4

# The course topics - Programming

This course assumes that you have little or no programming experience. It provides a solid background in good object-oriented programming techniques and introduces terminology using clear, familiar language. This course document includes materials for multiple lectures; however, only a subset—the most essential topics for achieving the learning outcomes—will be delivered during in-class sessions. The remaining slide decks, explanatory notes, and code examples will be published on Canvas for self-study. You are expected to review all posted materials, not just those presented live, as assignments and exam questions may draw on any content in the course set.

The following topics will be covered:

1. Introduction to the course
2. Eclipse IDE
3. Advanced Array Concepts
4. Advanced Inheritance Concepts
5. Revision control (optional)
6. Software project management with Maven (optional)
7. Exception Handling
8. File Input and Output (optional)
9. Database access
10. Threads and multitasking (optional)
11. User interfaces using JavaFX
12. Final exam guidelines

## 4.1 Introduction to the course

The first lecture in the semester will be used to introduce the goals, schedule, activities and literature for the *Object-oriented programming II* course.

### Material in Canvas

- OAP200V - Main document.pdf (this document)
- OAP200V-00-Introduction to the course.pdf

## 4.2 Eclipse IDE

This topic explains What Is Eclipse and How to Download, Install, and Configure Eclipse Workspace. It will also be explained how to Create a New Java Project and Run it as a Java Application in the Eclipse IDE

The Eclipse IDE (integrated development environment) provides strong support for Java developer. Eclipse is one of the leading IDEs with approximately one millions downloads per month.

Eclipse can be extended with additional software components called plug-ins. Pre-packaged Eclipse distributions provide a consistent set of functionality.

The Eclipse IDE for Java Developers distribution is designed to support standard Java development. It includes support for the Maven and Gradle build system and support for the Git version control system.

### Material in Canvas

- OAP200V-01-Eclipse IDE.pdf

### Online resources

- Eclipse tutorial: How to install and use Eclipse
- Java Hello World Example in Eclipse

### Tasks for individual work

1. Create Java project and run it as Java application.
2. Try debugging features of Eclipse (using *Debug as...* option).

## 4.3 Advanced Array Concepts

This topic discusses advanced array concepts, including array sorting techniques. Students will also learn about multidimensional arrays, and the Arrays and ArrayList utility classes. Finally, Chapter 9 explains how to create enumerations.

### Material in Canvas

- OAP200V-02-Advanced Array Concepts.pdf

### Tasks for individual work

1. Write an application containing an array of 15 String values (that are not originally in alphabetical order), and display them in ascending order. Save the file as StringSort.java.

2. a. Radio station KJAVA wants a class to keep track of recordings it plays. Create a class named Recording that contains fields to hold methods for setting and getting a Recording's title, artist, and playing time in seconds. Save the file as *Recording.java*.

3. b. Write an application that instantiates five Recording objects and prompts the user for values for the data fields. Then prompt the user to enter which field the Recordings should be sorted by—song title, artist, or playing time. Perform the requested sort procedure, and display the Recording objects. Save the file as *RecordingSort.java*.

4. Write an application that stores at least five different college courses (such as OAP200V), the time it first meets in the week (such as Mon 9 am), and the instructor (such as Johnson) in a two-dimensional array. Allow the user to enter a course name and display the corresponding time and instructor. If the course exists twice, display details for both sessions. If the course does not exist, display an error message. Save the file as *TimesAndInstructors.java*.

5. Create an application that contains an enumeration that represents the days of the week. Display a list of the days, and then prompt the user for a day. Display business hours for the chosen day. Assume that the business is open from 11 to 5 on Sunday, 9 to 9 on weekdays, and 10 to 6 on Saturday. Save the file as *DayOfWeek.java*.

## 4.4 Advanced Inheritance Concepts

This topic introduces students to more advanced inheritance topics. Students will learn to create abstract classes. Dynamic method binding enables

programmers to declare an object using the name of a superclass and then instantiate a subclass. A superclass can also be used as a method argument or as an array type. Students will learn about the Object class and how to use the *toString()* and *equals()* methods. Finally, students will learn to create interfaces and packages.

### Material in Canvas

- OAP200V-02-Advanced Inheritance Concepts.pdf

### Online resources

- Static and dynamic binding in java
- Choosing Between an Interface and an Abstract Class
- What is the Difference Between Abstraction and Inheritance
- Interface vs Abstract Class in Java: What's the Difference?

### Tasks for individual work

1. a. Create an abstract class named Book. Include a String field for the book's title and a double field for the book's price. Within the class, include a constructor that requires the book title, and add two get methods—one that returns the title and one that returns the price. Include an abstract method named *setPrice()*. Create two child classes of Book: Fiction and NonFiction. Each must include a *setPrice()* method that sets the price for all Fiction Books to $24.99 and for all NonFiction Books to $37.99. Write a constructor for each subclass, and include a call to *setPrice()* within each. Write an application demonstrating that you can create both a Fiction and a NonFiction Book, and display their fields. Save the files as **Book.java**, **Fiction.java**, **NonFiction.java**, and **UseBook.java**.

2. b. Write an application named BookArray in which you create an array that holds 10 Books, some Fiction and some NonFiction. Using a for loop, display details about all 10 books. Save the file as **BookArray.java**.

3. Create an abstract NewspaperSubscription class with fields for the subscriber name, address, and rate. Include get and set methods for the name field and get methods for the address and subscription rate; the *setAddress()* method is abstract. Create two subclasses named *PhysicalNewspaperSubscription* and *OnlineNewspaperSubscription*. The parameter for the setAddress() method of the PhysicalNewspaperSubscription class must contain at least one digit; otherwise, an error

message is displayed and the subscription rate is set to 0. If the address is valid, the subscription rate is assigned $15. The parameter for the *setAddress()* method of the *OnlineNewspaperSubscription* class must contain an at sign (@) or an error message is displayed. If the address is valid, the subscription rate is assigned $9. Finally, write an application that declares several objects of both subscription subtypes and displays their data fields. Save the files as **NewspaperSubscription.java**, **PhysicalNewspaperSubscription.java**, **OnlineNewspaperSubscription.java**, and **DemoSubscriptions.java**.

4. Create an abstract Student class for Parker University. The class contains fields for student ID number, last name, and annual tuition. Include a constructor that requires parameters for the ID number and name. Include get and set methods for each field; the setTuition() method is abstract. Create three Student subclasses named *UndergraduateStudent*, *GraduateStudent*, and *StudentAtLarge*, each with a unique *setTuition()* method. Tuition for an *UndergraduateStudent* is $4,000 per semester, tuition for a GraduateStudent is $6,000 per semester, and tuition for a StudentAtLarge is $2,000 per semester. Write an application that creates an array of at least six objects to demonstrate how the methods work for objects for each Student type. Save the files as **Student.java**, **UndergraduateStudent.java**, **GraduateStudent.java**, **StudentAtLarge.java**, and **StudentDemo.java**.

## 4.5 Revision control

Revision control (also known as version control, source control or source code management) is the management of multiple revisions of the same unit of information. It is most commonly used in engineering and software development to manage ongoing development of digital documents like application source code, art resources such as blueprints or electronic models, and other projects that may be worked on by a team of people. Revision control system or Version control system or Source code management stands for systems that allow software developers to manage application source code, including its various revisions.

### Material in Canvas

- OAP200V-04-Revision control.pdf

## 4.6 Software project management with Maven

Maven is an automation and management tool developed by Apache Software Foundation. It is written in Java Language and used to build and

manage projects written in C#, Ruby, Scala, and other languages. It allows the developer to create projects using Project Object Model and plugins.

It helps to build projects, dependency, and documentation. Maven is also able to build any number of projects into desired output such as jar, war, metadata.

### Material in Canvas

- OAP200V-05-Software project management with Maven.pdf

### Online resources

- Maven tutorial
- Using Maven with the Eclipse IDE
- Simple Explanation of Maven and pom.xml

### Tasks for individual work

1. Follow the guidelines from online resources section and check if maven installation for eclipse is present, install it if necessary
2. Follow the same guidelines (or the guidelines from lectures) and create a Java project with maven
3. Add dependency to the pom.xml file for *MySQL Connector/J* library

## 4.7 Exception Handling

This topic covers the important topic of error handling. In this chapter, students will learn to use the exception-handling mechanisms built into the Java language, including the try, catch, finally, and throws keywords. Students will learn to catch exceptions, obtain information about the errors, and find out about the different types of exceptions that may occur. The chapter concludes with a description of the assert statement, which can be used to verify the logical correctness of a program.

### Tasks for individual work

1. Write an application named *BadSubscriptCaught* in which you declare an array of eight first names. Write a *try* block in which you prompt the user for an integer and display the name in the requested position. Create a *catch* block that catches the potential *ArrayIndexOutOfBoundsException* thrown when the user enters a number that is out of range. The catch block also should display an error message. Save the file as **BadSubscriptCaught.java**.

2. The *Double.parseDouble()* method requires a String argument, but it fails if the String cannot be converted to a floating-point number. Write an application in which you try accepting a *double* input from a user and catch a *NumberFormatException* if one is thrown. The *catch* block forces the number to 0 and displays an appropriate error message. Following the catch block, display the number. Save the file as **TryToParseDouble.java**.

3. Write an application that displays a series of at least five student ID numbers (that you have stored in an array) and asks the user to enter a numeric test score for the student. Create a *ScoreException* class, and throw a *ScoreException* for the class if the user does not enter a valid score (less than or equal to 100). Catch the *ScoreException*, display an appropriate message, and then store a 0 for the student's score. At the end of the application, display all the student IDs and scores. Save the files as **ScoreException.java** and **TestScore.java**.

4. Write an application that displays a series of at least eight student ID numbers (that you have stored in an array) and asks the user to enter a test letter grade for the student. Create an *Exception* class named *GradeException* that contains a static public array of valid grade letters (A, B, C, D, F, and I) that you can use to determine whether a grade entered from the application is valid. In your application, throw a *GradeException* if the user does not enter a valid letter grade. Catch the *GradeException*, and then display an appropriate message. In addition, store an I (for *Incomplete*) for any student for whom an exception is caught. At the end of the application, display all the student IDs and grades. Save the files as **GradeException.java** and **TestGrade.java**.

## 4.8 File Input and Output

This topic introduces students to input and output in Java. Students will discover that files are treated as streams of bytes. The Java language provides many built-in classes for working with files. Students will learn about sequential access and random access files, and how to work with each of them.

### Tasks for individual work

1. Create a file using any word-processing program or text editor. Write an application that displays the file's name, containing folder, size, and time of last modification. Save the file as **FileStatistics.java**.

2. Create three files of any type you choose—for example, word-processing documents, spreadsheets, or pictures. Write an application that determines whether the first two files are located in the same folder as the third one. Test the program when the files are in the same folder and when they are not. Save the file as **CompareFolders.java**.

3. Create a file that contains a line of your favorite song lyric. Use a text editor such as Notepad, and save the file. Copy the file contents, and paste them into a word-processing program such as Word. Write an application that displays the sizes of the two files as well as the ratio of their sizes to each other. Save the file as **FileSizeComparison.java**.

4. Write an application that determines which, if any, of the following files are stored in the folder where you have saved the exercises created in this chapter: *autoexec.bat*, *CompareFolders.java*, *FileStatistics.class*, and *Hello.doc*. Save the file as **FindSelectedFiles.java**.

## 4.9 Database access

In this topic, you learn how to connect to a MySQL database using Java JDBC. The course starts with an overview of the JDBC API. Then you learn how to set up your development environment with the appropriate MySQL database drivers. Next the course shows you how to submit a SQL query and process the result set. Also, you learn how to perform SQL insert, updates and deletes. The course moves on to advanced topics such as Prepared Statements to handle SQL paramaters.

### Material in Canvas

- OAP200V-08-Database access.pdf
- Sample database
  - mysqlsampledatabase.zip
  - MySQL Sample Database guidelines. This online tutorial uses the code stored in jdbc-source-code-v2.zip

### Online resources

- Java Database Connection: JDBC and MySQL
- MySQL Java tutorial
- Hibernate MySQL Maven Hello World Example
- Use Hibernate Tools to generate Entity classes from Tables
- Top 10 JDBC Best Practices for Java Programmer

## 4.10 User Interface

This topic covers creating user interfaces using JavaFX. JavaFX is the modern toolkit for developing rich graphical user interfaces in Java and is the recommended alternative to the older Swing library. While basic GUI concepts have been introduced in earlier courses, this module provides a deeper look at user interface design using JavaFX.

Students will learn to use *Stage*, *Scene*, *Label*, *TextField*, *Button*, *CheckBox*, *ToggleGroup*, and *ComboBox* components. Rudimentary event handling is covered with an emphasis on *EventHandler* and lambda expressions.

### Objectives

- Describe JavaFX architecture and components
- Use the *Stage* and *Scene* classes
- Use layout containers like *VBox*, *HBox*, and *GridPane*
- Add *Label*, *TextField*, *Button*, *CheckBox*, and *ComboBox* components to a scene
- Understand and implement event-driven programming using JavaFX
- Use *EventHandler* and lambda expressions for event handling
- Use *ToggleGroup* for grouping *RadioButton* controls

### Material in Canvas

- OAP200V-10-Introduction to JavaFX Components.pdf

### Online resources

- Official JavaFX Documentation
- JavaFX API Documentation
- JavaFX Tutorial by Code.Makery
- JavaFX on Stack Overflow

### Tasks for individual work

1. Create a JavaFX application that asks the user for their favorite color. The application should display a *ComboBox* with at least five predefined color options and allow the user to enter a custom color.

2. Enhance the application from the previous task by adding an event handler that responds to changes in the ComboBox. When the user selects or enters a color, display a dialog (e.g., using *Alert*) showing their choice.

3. Create a JavaFX interface for ordering a pizza. The user should be able to select toppings, size, and sauce, and provide a delivery address. Calculate and display the cost of the pizza based on selected options.

4. Write a JavaFX application that displays the opening sentence or two from your favorite book in a *Label*. Save the file as *BookQuote.java*.

5. Add a button to the frame in the *BookQuote* application. When clicked, it should display the title of the book in a separate label or dialog. Save as *BookQuote2.java*.

6. Write a JavaFX application with a *Button* that becomes disabled after it is clicked. Save the file as *DisableButton.java*.

7. Modify the *DisableButton* program so that the button disables only after the user clicks it eight times. Then, display a label with the text "That's enough!" Save as *DisableButton2.java*.

8. Create a JavaFX application with six labels showing facts about a topic of your choice. Each time the user clicks a button, replace one of the current labels with a new one. Save as *Facts.java*.

9. Develop an application for Lambert's Vacation Rentals using JavaFX. Include controls to select one of three locations, number of bedrooms, and an option to include meals. Compute and display the total cost. Save as *VacationRental.java*.

10. Design a JavaFX application for the *Sublime Sandwich Shop*. The user selects a main ingredient and bread type from combo boxes or list views. Display the calculated price. Save as *Sandwich.java*.

11. Write an application where the user selects a country from a list. Display the capital city of the selected country. Save as *Capitals.java*.

## 4.11 Threads and multitasking

Multithreading in Java is a process of executing multiple threads simultaneously. A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

This section presents just a brief overview of the concepts.

### Material in Canvas

- OAP200V-10-Threads and multitasking.pdf

## 4.12 Final exam guidelines

Final guidelines are given within this topic along with the overview of Object-oriented design principles that every students should have in mind during programming.

### Material in Canvas

- OAP200V-11-Object-oriented design principles.pdf
- OAP200V-11-Final exam guidelines

# 5

# Mandatory Assignment

The details will be given later

# 6

# The running example

## 6.1 Problem Description: Cinema Manager

### Context

Modern cinemas require a flexible and efficient software system to manage daily operations, including movie scheduling, customer reservations, and reporting. The system must support multiple movie halls screenings in the cinema, a dynamic movie catalog, and customer service operations performed by staff.

### Problem Statement

The management of the cinema seeks to replace its current manual and fragmented systems with a unified desktop application. This application should allow cinema staff to manage movie screenings, customer reservations, movie offers, and customer data. The system must support multiple movie halls and provide real-time access to scheduling, film data, and customer information.

### Stakeholders

- **Cinema staff:** Responsible for managing screenings, reservations, and customer data.
- **Customers:** Interact with the system indirectly through the staff for ticket reservations.
- **Managers:** Require access to reports such as attendance and screening popularity.

### System Goals

- Manage a catalog of movies with associated genres, languages, durations, and descriptions.
- Schedule movie screenings across various cinema halls.
- Register customers and manage their reservations.
- Provide accurate and real-time movie and screening information.
- Generate operational reports including upcoming screenings, sold-out shows, and attendance statistics.

### Key Functionalities

- Add, edit, and remove movies from the catalog.
- Schedule and manage screenings for specific dates, times, and cinema halls.
- Register and manage customer information.
- Display detailed information about movies and screenings.
- Generate management reports.

### System Environment

- Desktop application developed using Java (JavaFX)
- MySQL relational database backend
- JDBC for database connectivity

## 6.2 Requirements Analysis

### Actors

- **Staff Member** – A cinema employee who uses the application to manage movies, screenings, reservations, and customers.
- **Manager** – A senior staff user who accesses additional reporting functionality.
- **Customer** – An individual who reserves tickets through staff (interacts indirectly with the system).

### Use Cases

The system must support the following primary use cases:

- **UC1: Manage Movies**
  Add, edit, and remove movie entries from the catalog (title, duration, genre, language, etc.).

- **UC2: Schedule Screenings**
  Create and update scheduled showings for specific movies, times, and halls.

- **UC3: Register Customer**
  Add a new customer to the system with contact details.

- **UC4: Make Reservation**
  Reserve a ticket for a specific screening on behalf of a customer.

- **UC5: Generate Reports**
  Produce statistics such as attendance rates, most popular films, etc. (Manager only).

### Functional Requirements (tabulated)

Table 6.1: Functional Requirements (FR)

| ID | Requirement | Priority |
|---|---|---|
| FR1 | The system shall allow staff to create, edit, and delete movie entries. | High |
| FR2 | The system shall store metadata for each movie (title, description, genre, language, duration). | High |
| FR3 | The system shall allow scheduling of screenings, specifying the movie, time, and hall. | High |
| FR4 | The system shall prevent overlapping screenings in the same hall. | High |
| FR5 | The system shall allow staff to register customers and store their contact information. | High |
| FR6 | The system shall enable movie reservation and associate it with customers. | High |
| FR7 | The system shall display a searchable and filterable list of Movies. | Medium |
| FR8 | The system shall allow managers to view summary reports (e.g., movie reservations, most-viewed movies). | Medium |

Table 6.2: Non-Functional Requirements (NFR) with category

| ID | Requirement | Category | Priority |
|---|---|---|---|
| NFR1 | The application shall be developed in Java using JavaFX for the user interface. | Constraint (Design) | Must |
| NFR2 | The system shall store and retrieve data using a MySQL database via JDBC. | Constraint (Implementation) | Must |
| NFR3 | The application shall provide a responsive and intuitive user interface for staff. | Usability | Should |
| NFR4 | All inputs shall be validated for completeness and correctness. | Accuracy | Must |
| NFR5 | The system shall support up to 5 concurrent users with an average response time of less than 2 seconds for database-backed operations (e.g., reservations, scheduling). | Performance | Should |

**Non-Functional Requirements (tabulated)**

## 6.3 Structure of the Code

To ensure the *Cinema Manager* application is modular, maintainable, and scalable, the codebase is organized into logical packages with clear responsibilities and one-way dependencies. The following scheme shows the code structure:

```
com.cinemamanager
|
+-- MainApp.java        // JavaFX entry point
+-- controller          // JavaFX controllers (UI orchestration)
+-- model               // Domain classes (Movie, Hall, Screening, Customer, Reservation
+-- dao                 // Data access objects (JDBC to MySQL)
|    +-- ...Dao         // Interfaces
|    +-- ...DaoImpl     // JDBC implementations
+-- service             // Business logic and validation (e.g., overlap rules)
+-- util                // Utilities (DB connection, alerts)
+-- view                // JavaFX views (manually built UI components)
```

To ensure clarity and maintainability of the Cinema Manager application, the codebase follows a layered architecture with clear separation of concerns. Each package and component has a well-defined responsibility, avoiding duplication and reducing coupling. The following rules summarize the main principles applied in the design:

- **View** contains only UI widgets and formatting. No database access.




Figure 6.1: Use Case Diagram for the Cinema Manager system

- **Controller** handles user events, reads/writes view state, and calls Service/DAO.

- **Service** holds reusable business rules (e.g., computing end-time, overlap checks).

- **DAO** is responsible for persistence only. Controllers/Services depend on DAO *interfaces*, not implementations.

- **Util** provides supporting infrastructure (e.g., database connection, alerts).

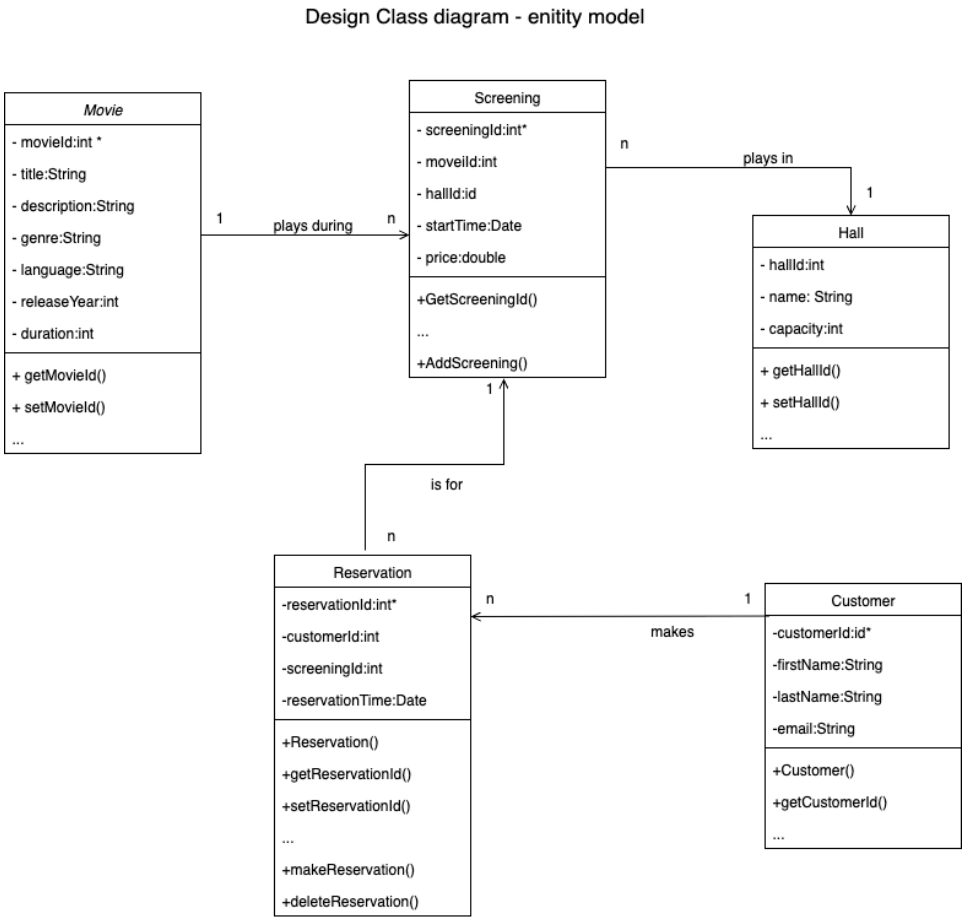



Figure 6.2: Design Class Diagram for the Cinema Manager system

## Package Purpose

| Package | Purpose |
|---|---|
| `controller` | Contains JavaFX controller classes like `MovieController`, `ScreeningController`, etc., responsible for handling UI actions and linking the view with the business logic. |
| `model` | Defines core domain entities such as `Movie`, `Customer`, `Screening`, `Hall`, and `Reservation`. These are used throughout the application to represent business data. |
| `dao` | Manages data persistence and retrieval. Classes like `MovieDAO` and `CustomerDAO` encapsulate all database operations. |
| `service` | Implements business logic, including validation (e.g., checking for overlapping screenings). It acts as a layer between the controller and DAO layers. |
| `util` | Contains utility classes for database connections, date/time handling, and other shared functionality. |
| `view` | Stores FXML layout files (if used) or programmatic UI definitions. |

Table 6.3: Package responsibilities in the Cinema Manager application

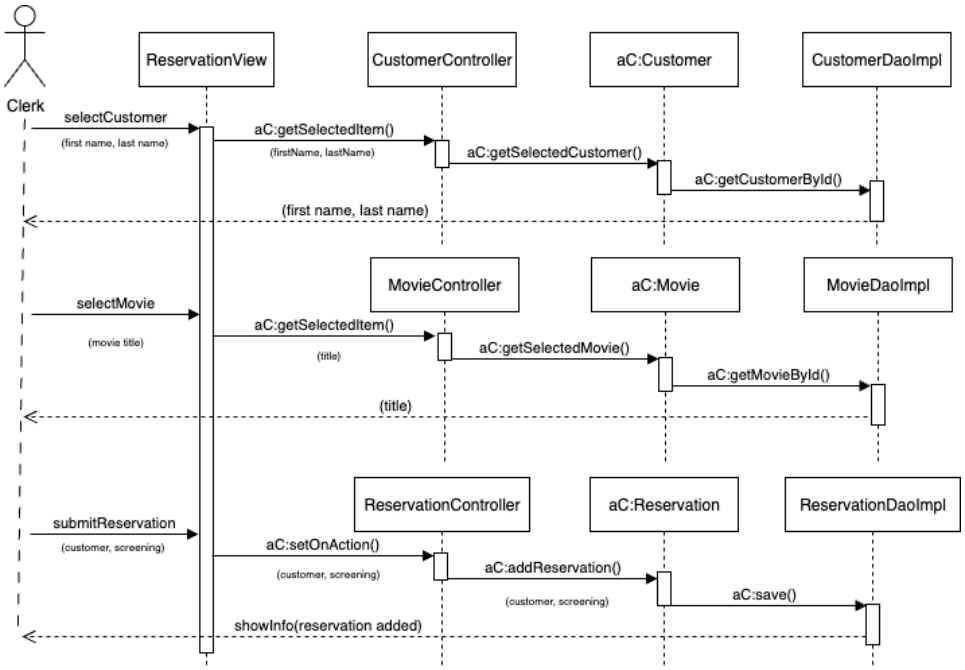



Figure 6.3: Sequence Diagram for the Cinema Manager system

## Mapping to Functional Requirements

| Requirement | Relevant Packages | Example Classes |
|---|---|---|
| 1–2 | `controller`, `model`, `dao` | `MovieController`, `MovieDAO`, `Movie` |
| 3 | `controller`, `dao`, `model` | `SchedullingController`, `ScreeningDAO`, `Hall`, `Screening` |
| 4 | `service`, `dao`, `model` | `ScreeningService`, `ScreeningDAO`, `HallDao` |
| 5 | `controller`, `model`, `dao` | `CustomerController`, `CustomerDAO`, `Customer` |
| 6 | `controller`, `model`, `dao` | `ReservationController`, `ReservationDAO` |
| 7 | `controller`, `model`, `dao` | `MoviesController`, `MovieView` |
| 8 | `controller`, `view` | `appController`, `MainView` |

Table 6.4: Mapping of functional requirements to packages and classes

# 7

# The exam

For the final exam, your team is required to develop a comprehensive Java desktop application tailored to a project description, presented in Chapter chapter 8.

## 7.1 Your task

The project-based examination is designed to evaluate students' comprehensive understanding and practical application of software development concepts and skills in the context of designing, implementing, and presenting a functional app. This exam consists of two distinct parts that together form the basis for grading:

1. Written exam (submitted in WISEFlow)
    - Problem analysis and system design report (Part 1.)
    - Implementation of the application (Part 2.)
2. Oral exam (live demo)
    - Final presentation demo (Part 3.)

These parts will be explained in detail in the following section.

### 7.1.1 Part 1: Problem Analysis and System Design

In this phase, each team is expected to write a comprehensive report that outlines the analysis and design of the app based on a provided problem statement. This part assesses students' ability to analyze a problem, conceptualize an effective design, and articulate their ideas in a written format. Therefore, individual contributions from each group member must be specified within the report, outlining their specific roles and responsibilities.

The teams are expected to execute and document the following activities:

1. Problem Analysis
    - **Analyze the problem.** Conduct an in-depth analysis of the requirements based on the project description, pinpointing crucial functionalities and their interconnections. Define the problem statement and the product scope.
    - **Identify functional requirements**. Define and outline the core features and capabilities that the system must have to meet user needs.
    - **Identify non-functional requirements**. Establish important properties and restrictions of the system.
2. UML design
    - Utilize UML diagrams, i.e., use-case, design class, and sequence diagrams to visually represent the architecture and interactions of the system. In addition, identify and list the most important use cases.
    - Clearly define relationships between classes, interfaces, and system components.
    - Present data flow and user interactions.
    - Define diagrams that support user needs and the functional and non-functional requirements identified in the analysis part.
3. Application Presentation
    - The report shall include an overview of the user interface, illustrating how users interact with the system to perform tasks.
    - Highlight the key features that were implemented according to the functional requirements (FRs), explaining how they are realized in the system.
    - If some features are not fully implemented, explain their current status, planned design, and potential next steps.
    - Connect the theoretical design (UML diagrams, requirements analysis) with the practical implementation in your code.
4. Testing Insights and Challenges
    - Discuss the testing efforts you undertook to ensure the application's correctness and robustness.
    - Share any insights gained from your testing process and how they influenced your design choices.
    - Address any challenges you encountered during the implementation and testing phases and how you mitigated them.

5. Documentation Overview

    - Summarize the key components of your written report, highlighting the problem analysis, design justifications, and key insights.
    - Explain how your documentation supports the design and implementation choices you made.
    - Each student individually reflects on their work, skills, and knowledge they gained from the project.
    - Be ready to answer any questions the examiners may have about the contents of your documentation.

### 7.1.2 Part 2: Implementation of the app

Once the analysis and design phase is complete, teams must implement the app based on the design specifications outlined in their report. The implementation should adhere to industry-standard coding practices, including proper code organization, documentation, and adherence to design principles. Collaborative coding techniques are encouraged, but individual contributions must be discernible within the codebase. This part evaluates students' coding proficiency, teamwork, and attention to detail in translating design concepts into a functional application.

1. Java Application

    - Develop the Java desktop application based on your analysis and design.
    - Structure your codebase using appropriate packages for different functions.
    - Implement file system interactions for storage and utilize MySQL database interactions for manipulating data.

2. User Interface (UI)

    - Design a user-friendly graphical user interface (GUI) that guarantees intuitive navigation and interaction with the application's features. It is permitted to use both JavaFX and Swing for this purpose.
    - Ensure each primary function has dedicated UI components for easy access.

3. Documentation

    - The eam must generate technical documentation of the code using Javadoc. This involves writing clear and consistent in-code

comments for classes, methods, and attributes, and then producing HTML documentation pages with the Javadoc tool. The generated pages should be submitted together with the code, as they provide a professional reference for the system's API and implementation details.

4. Testing and Source Code

   - Thoroughly test the application to ensure functionality, user-friendliness, and resilience.
   - Submit the complete source code of the developed application.

### 7.1.3 Part 3. Final presentation demo

After successfully implementing the app, each team is required to provide a live demonstration of the application. During the demonstration, teams are expected to showcase the app's functionality, user interface features, and how it addresses the problem statement. The live demonstration provides an opportunity for students to showcase their communication skills, demonstrate the usability of their application, and demonstrate their ability to handle real-time questions and challenges.

Each team will be allocated a specific time slot (30 minutes - 20 for presentation, 10 for questions) for their oral exam (each group member should have their role in the presentation). During this time, your team will be expected to deliver a comprehensive presentation that encompasses the following elements:

- **Part 1** - Problem Analysis and Design Justification
  - Clearly articulate the core problem defined in and the rationale for your chosen application design.
  - Explain your design decisions, highlighting how they address the project description.
  - Elaborate on the significance of the UML diagrams and how they capture the essence of the system's structure and interactions.
- **Part 2** - Implementation and Demonstration
  - Showcase your developed Java desktop application to the examiners.
  - Navigate through the application's features, demonstrating the user interface and functionality.
  - Highlight the integration with the database and the file system interactions.

  - Illustrate how the application addresses the company's needs and streamlines its operations.

Project presentations will be held during a scheduled time slot around the final exam, and the actual date will be determined by the examination office.

## 7.2 Java app requirements

The exam aims to evaluate students' abilities to work in teams and develop a complete application enabling users to browse and edit data stored in the required sample MySQL database. The team has the freedom to design and implement the application and its features based on the identified requirements defined in the next section.

### 7.2.1 Minimum requirements (for minimum passing grade)

The resulting application should include the following elements:

- The main view that enables a user to execute the required features.
- Connection and access to a MySQL database.
- Generated detailed Java documentation (using JavaDoc)
- Every submitted class must contain details about the author(s), their contribution, the purpose of the class, and an explanation of the included methods.

### 7.2.2 Optional requirements (for higher grade)

In addition to the minimum requirements, the submitted application may include additional features. It is up to each team to decide which features to implement. For the maximum grade, the application should demonstrate well-defined use cases based on identified requirements and design. Possible examples include:

- Accessing, reading, and writing to the file system.
- Providing a separate interface for entering or modifying existing records.
- Designing a more advanced main view with additional interface elements to improve usability and presentation.

Beyond these suggestions, the application may also include additional features that further enhance the overall user experience. The specific scope and creativity of the solution are left to the students' imagination.

The developed application needs to be prepared for further development. Therefore, it is essential that the code is of high quality, easy to understand, and possesses certain qualities, such as high cohesion, low coupling, and strong encapsulation. The solution is assessed according to the team's ambitions and how well they have been fulfilled.

### 7.2.3 Specific requirements

The code must be written in Java using the Java Standard Edition (Java SE). The graphical user interface (GUI) can be implemented using JavaFX or Swing. All dependencies on external Java code should be resolved using Maven. If you want to try to solve the tasks using external libraries or other known tools, you must organize the project as a Maven project. You should let Maven solve all the problems associated with dependencies.

The application should implement a connection to a MySQL database. It is up to the students to decide whether they want to use the existing database or design their own. Details about the suggested database, its structure, and installation are presented in section 8.2. In the case of their database, the team should submit the SQL script that creates and populates the database.

The entire project must be packaged as an Eclipse project in a zip file. It must be exported and packaged so the zip file can later be imported into Eclipse again at the root level of an existing project. The team must carefully test that the import is possible and easy, and that no additional customization is needed. Perform tests on multiple machines and multiple Eclipse instances. Here, you can find out how to wrap the project so that it can be easily and quickly reopened as a project elsewhere. Link to the film: `https://youtu.be/rbfblRzwT0s`

If you did not create a Maven project because you did not use external Java code or external tools, do not include the pom.xml file shown in the movie. If you have created a Maven project, pom.xml must be included in the submission. Any resource files must only be included if the application cannot function without them. The application should be able to generate necessary resource files if they do not exist.

Submit only one zip file with the project source files, containing a separate docs folder with generated documentation. There is no need to include the database in the submission.

## 7.3 Evaluation

During the academic year, students will participate in teamwork that involves developing an application. The **Submission** (including the program code and written report) and a successful **live demo** (oral exam) form the basis for the assessment (i.e., final grade) in the course.

### 7.3.1 Submission

Each team is required to submit three-part submissions on the exam date:

1. Part 1 - Problem Analysis and System Design (single pdf file):
    - Written report containing thorough problem analysis, UML diagrams, and Architecture Model illustrations (including individual contributions of each team member). The detailed report requirements are presented in subsection 7.1.1.
2. Part 2 - Implementation of the app (single zip file):
    - Source code of the Java application.
    - Detailed report encompassing problem analysis justification, design explanation, and testing insights.
    - Comprehensive user manual for the application.
3. Part 3 - Presentation used at the final presentation demo.

## 7.4 Grading Criteria

The following grading criteria will be used for

1. Written report
    - Clear articulation of the problem statement and system boundaries.
    - Quality and coherence of UML diagrams.
    - Completeness, relevance, and correctness of listed FRs and NFRs
    - Clarity and usability of the described or implemented UI.
    - Structure, clarity, academic style, and coherence of the report.
2. Delivered app
    - Adherence to requirements and overall design
    - Correct implementation and UI design.
    - Clarity, completeness, and cohesiveness of the documentation.
    - Thoroughness and success of testing endeavors.
    - All requirements and restrictions described above must be followed exactly.
    - The final grade depends on the complexity and quality of implemented features.

- The code must have high cohesion and low coupling at both class and package levels.
- Class members (variables or methods) must have strong encapsulation and good semantics (name-giving). They should be the same as the ones presented in the Class Diagram.
- All run-time errors should be successfully handled.
- The submitted zip file should be easily imported and used at the root level in a project in Eclipse without any problems.
- The user interface should be of high quality and intuitive.
- The code should be well-documented, readable and maintainable.

3. Live demo

   - Your oral exam will be evaluated based on the clarity, depth, and coherence of your presentation, as well as your ability to effectively communicate your design choices and implementation efforts. The examiners will also assess your understanding of the application's domain and its alignment with real-world requirements.

Students are assessed individually based on their contribution to the overall code. Therefore, it is very important to document your code and state the authors' contributions in each class.

## 7.5 Important notes

Please keep in mind the following things:

- Detailed guidelines on the structure and the required contents of the report can be found in this document published in Canvas.
- Keep the report brief and up-to-the-point. The best way to achieve that is to avoid long textual explanations and definitions, and provide more structured content. For example, you can use tables to show functional and non-functional requirements, their numbering (FR1, FR2, ..., NFR1, NFR2,...), information on their priority, which team member has been responsible for their implementation, etc.
- Regard the report as a technical document intended for the stakeholders familiar with the terminology and technical stuff. For example, **you don´t need to provide definitions of the specific diagrams or explain their purpose**. By designing and using the diagrams correctly, you will demonstrate your knowledge and mastery of the software design.

- The report should not exceed 40 pages.

By presenting your work in a clear and understandable way, you will help us understand your effort and prevent us from missing important information about your system.

## 7.6 Problem Analysis and System Design Report

This report documents the analysis, design, and development process of the exam project. It serves as both a technical record and an evaluation artifact, showing how the team has translated a problem statement into a working Java application. The report demonstrates skills in problem analysis, requirement specification, UML modeling, database design, implementation, and testing. In addition, it highlights teamwork and individual contributions, ensuring that each member's role is clearly identified. The structure below is suggested for the final submission.

### 7.6.1 Introduction

- Introduce the project and its overall purpose.
- Present the chosen domain (e.g., cinema, DVD store, streaming service).
- Define the scope of the system and the intended users.

### 7.6.2 Problem Analysis

**Problem Statement**

- Describe the main problem the application is solving.
- Identify limitations of existing solutions (manual or fragmented processes).

**Functional Requirements (FRs)**

- List all required system functionalities (CRUD operations, reporting, etc.).
- Prioritize them into essential and optional features.

**Non-Functional Requirements (NFRs)**

- Performance, usability, security, reliability, maintainability.
- Technical constraints (Java SE, MySQL, Maven, JavaFX/Swing).

### 7.6.3 System Design

**Use Case Modeling**

- Use case diagram of the system.
- Brief descriptions of the main use cases.

**Domain Model and Class Design**

- Class diagram showing entities, associations, and attributes.
- Design choices: cohesion, coupling, encapsulation.

**Sequence Diagrams**

- Illustrate dynamic interactions for 2–3 key use cases.

**Architecture Model (4+1 Views)**

- Logical View (main packages and classes).
- Development View (code organization, Maven structure).
- Process View (runtime behavior, DB connections).
- Physical View (deployment environment).
- Scenarios (show how views support use cases).

### 7.6.4 Database Design

- ER diagram of the database (provided schema or self-designed).
- Description of the four main entities (student/teacher/course/exam or Sakila DB tables).

### 7.6.5 User Interface Design

- Mockups or screenshots of the planned UI.
- Navigation flow and usability considerations.

### 7.6.6 Implementation Overview

- Mapping of requirements to implemented features.
- Description of important packages, classes, and methods.
- Summary of JavaDoc documentation structure.

### 7.6.7 Testing Insights

- Test strategy (unit tests, manual functional testing).
- Description of tools used (JUnit, etc.).
- Key challenges faced and how they were solved.

### 7.6.8 Challenges and Limitations

- Technical or conceptual difficulties encountered.
- Features not implemented and planned next steps.

### 7.6.9 Individual Contributions

- Table mapping team members to tasks and responsibilities.
- Short personal reflections on learning outcomes.

### 7.6.10 Conclusion

- Summary of what was achieved.
- Reflection on the link between analysis, design, and implementation.
- Potential improvements for future iterations.

# 8

# Project description for the exam

## 8.1 Problem: Private Book Library Manager

### Context

Many people own books that are spread across different rooms, bookshelves, boxes, or storage locations. Over time, it can become difficult to remember which books are already owned, where they are placed, which books have been read, and which books belong to a certain writer or genre. A lightweight desktop application can help private users organize and search their personal book collection in a structured way.

### Problem Statement

The user wants to develop a desktop application for managing a private book library. The system should allow the user to register books, writers, genres, and bookshelf locations. It should also support tracking whether a book has been read or not, and make it easy to search and review the collection. The application should provide a clear overview of the books owned by the user, including where each book is located and which books belong to specific writers or genres.

### System Goals

- Maintain a registry of books in a private collection.
- Store information about writers, genres, and bookshelf locations.
- Track whether a book has been read or not.
- Provide quick access to information about owned books.

- Support searching and filtering by book title, writer, genre, bookshelf, and reading status.
- Generate simple reports about the private library collection.

### Key Functionalities

- Add, edit, and remove books from the private library.
- Register and manage writers, genres, and bookshelves.
- Assign each book to a writer, genre, and bookshelf.
- Mark books as read, unread, or currently being read.
- Display detailed information about each book.
- Search for books by title, writer, genre, or bookshelf.
- Show all books written by a specific writer.
- Show all books placed on a specific bookshelf.
- Generate summary reports, such as books by genre, books by writer, read books, unread books, and total number of books in the collection.

### System Environment

- Desktop application developed in Java (JavaFX)
- MySQL relational database backend
- JDBC for database connectivity

## 8.2 Suggested Database

For this project, we suggest a simple relational database consisting of four tables: `book`, `writer`, `genre`, and `bookshelf`. Students may use this proposed schema as a starting point, or they may design and implement their own database model that fulfills the project requirements.

- **book:** Stores information about books owned by the user, such as ID, title, publication year, reading status, and references to writer, genre, and bookshelf.
- **writer:** Stores information about writers, such as ID, name, nationality, or other relevant information.

- **genre:** Stores information about book genres, such as ID, name, and description.
- **bookshelf:** Stores information about bookshelf locations, such as ID, name, room, shelf number, or other location details.

The relationship model is straightforward:

- A writer can have many books, but each book is written by one writer.
- A genre can include many books, but each book belongs to one main genre.
- A bookshelf can contain many books, but each book is placed on one bookshelf.
- Each book record links a specific book to its writer, genre, and bookshelf location.
- Each book should also store information about its reading status, for example whether it is read, unread, or currently being read.

# 9

# Uses of Java Programming

Java is an OOPs (Object-Oriented Programming) language. A team of Sun Microsystems engineers called the "Green Team" created Java in 1991. It was first released in 1995, and since then, several new versions have been launched.

Programs developed in Java will be executed in a JVM (Java virtual machine) by which we can run the same program on multiple platforms. Many of the applications that make day-to-day living easier are built with Java.

## Android app

Java is the standard language of Android app development, which means the fastest performance and most native experience.

## Desktop Apps

Java is probably the easiest way to create cross-platform software these days. The Swing UI toolkit was once an alternative for Java developers, but if you're starting now, consider skipping it and learning JavaFX instead.

## Enterprise systems

High-volume data processing industries (such as banking, financial trading, etc.) use Java for non-legacy systems because it is fast, portable, easy to maintain, and is less prone to types of catastrophic bugs in low-level languages.

## Embedded Systems

Some areas of embedded space now depend on Java. Examples of embedded systems include digital watch, factory controller, traffic lights, microcontrollers, hybrid vehicles, and more recently, Internet of Things devices.

## Scientific research

Java is primarily used for complex tasks such as natural language processing and artificial intelligence. Data processing, computing, modeling, and simulations are often carried out in languages such as MATLAB and Python.

# 10

# Software

This chapter contains an explanation of how to download and install the two most important tools that you need to get started. These include the **Java Development Kit - JDK** and whatever **integrated development environment - IDE** you plan on using.

## Java Development Kit - JDK

The Java Development Kit, or JDK, is a program that allows you to write Java code from the comfort of your desktop. It contains a variety of tools that are very useful for creating, running, and optimizing your Java code. An integrated development environment (IDE) can be used to make writing code easier than ever before.

Before you can use the JDK, you will need to download and install it on your computer:

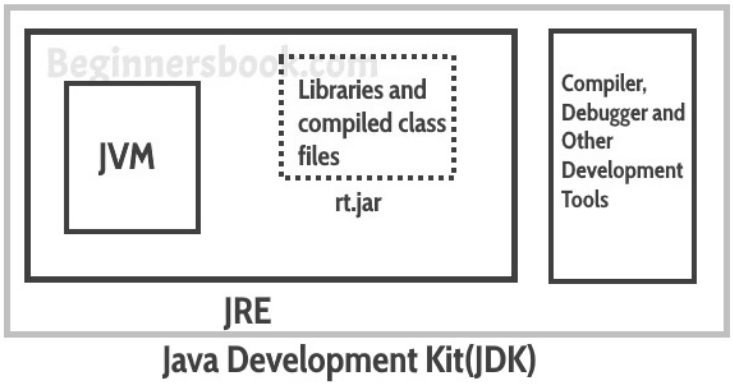
1. Head to the **Java SE Downloads page** and click on the version of the JDK you would like to install (JavaSE, version 22). There you will find different versions of Java. Choose and download the one compatible with your operating system. Follow the prompts to begin downloading the right package for your operating system.

2. Once the download is complete, you need to run it. The best way to do this will depend on whether you're using Windows or Mac, but it won't be any different from installing any other piece of software.

3. Follow the prompts to install the program and wait for it to complete.

For detailed instructions for setting up Java, refer to this **article**.

If you need a short clarification of the terms (JDK, JRE, JVM) check the article **here**




## Development environment

A number of integrated development environments (IDEs) are available free of charge for various operating systems. These IDEs offer a variety of features: building Java applications, debugging, code inspections, code assistance, JUNIT testing, multiple refactoring, visual GUI builder and code editor, Java, Maven build tools, data modeling and build queries, and more.

- **BlueJ** is an integrated development environment (IDE) for the Java programming language. It was created specifically for beginning Java students and is available for MacOS, Windows, and other systems. BlueJ requires that you already have Java installed. However, it is also suitable for those who wish to do small-scale software development. It runs with the help of a JDK (Java Development Kit). It can run on Windows, Mac OS X, Linux and other platforms which run Java. Download BlueJ from **here**.

- **NetBeans** is an open-source Integrated Development Environment written in Java.

  The NetBeans IDE can be used to develop in Java, but also supports other languages. These include PHP, C/C++, and HTML5. NetBeans is cross-platform and runs on Microsoft Windows, Mac OS X, Linux, Solaris and other platforms supporting a compatible JVM. It can also be used for working with Cloud applications.

  Download NetBeans from **here**.

- **Eclipse** is another free Java IDE for developers and programmers. It is mostly written in Java. Eclipse lets you create various cross-platform Java applications for use on mobile, web, desktop and enterprise domains.

  It also contains a base workspace with an extensible plug-in system for customizing the IDE to suit your needs. Through plugins, you can

develop applications in other programming languages. These include C, C++, JavaScript„ Perl, PHP, Prolog, Python, R, Ruby (including Ruby on Rails framework), to name just a few.

Eclipse is available under the Eclipse Public License and is available on Windows, Mac OS X and Linux. Download eclipse from **here**.

- **IntelliJ IDEA** is an integrated development environment (IDE) written in Java for developing computer software written in Java, Kotlin, Groovy, and other JVM-based languages. It is developed by JetBrains (formerly known as IntelliJ) and is available as an Apache 2 Licensed community edition, and in a proprietary commercial edition. Both can be used for commercial development.

# 11

# Detailed guidelines for setup

## 11.1 Setting Up JavaFX + Maven Project in Eclipse

This section outlines the steps for setting up a JavaFX desktop application using Maven in Eclipse IDE. Instructions are provided separately for Windows and macOS platforms.

### Overview of Required Steps

1. Install Eclipse IDE for Java Developers
2. Use Eclipse's built-in Maven and JDK
3. Download and extract JavaFX SDK
4. Create a new Maven project in Eclipse
5. Add JavaFX and MySQL JDBC dependencies to `pom.xml`
6. Create a simple JavaFX main class
7. Configure and run the application via Maven

### 11.1.1 1. Windows Setup

#### 1.1 Install Eclipse IDE

- Download Eclipse IDE for Java Developers from `https://www.eclipse.org/downloads/`
- During installation, choose the Eclipse bundle that includes:
  - Java 17 or higher (JDK)
  - Apache Maven

### 1.2 Download JavaFX SDK

1. Go to `https://openjfx.io`
2. Download the JavaFX SDK (e.g., version 21)
3. Extract it to a folder, e.g.:

   ```
   C:\JavaFX\javafx-sdk-21
   ```

### 1.3 Create Maven Project

1. In Eclipse: `File > New > Maven Project`
2. Choose archetype: `maven-archetype-quickstart`
3. Set Group ID: `com.cinemamanager`, Artifact ID: `cinema-manager`

### 1.4 Add JavaFX and MySQL Dependencies

- Open the generated `pom.xml`
- Inside the `<dependencies>` tag, add:
  - JavaFX Controls and FXML:

    ```
    org.openjfx:javafx-controls:21
    org.openjfx:javafx-fxml:21
    ```

  - MySQL Connector/J:

    ```
    com.mysql:mysql-connector-j:8.4.0
    ```

### 1.5 Create JavaFX Main Class

1. Navigate to `src/main/java`
2. Create the package `com.cinemamanager`
3. Create the file `MainApp.java` with the following content:

```
package com.cinemamanager;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.stage.Stage;
```

```
public class MainApp extends Application {
    @Override
    public void start(Stage stage) {
        Label label = new Label("Hello, Cinema Manager App!");
        Scene scene = new Scene(label, 400, 200);
        stage.setTitle("Cinema Manager");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

### 1.6 Run JavaFX Project

1. Go to `Run > Run Configurations...`
2. Right-click `Maven Build > New Configuration`
3. Set:
   - Name: `Run JavaFX`
   - Base directory: your project folder
   - Goals: `clean javafx:run`
4. Click `Apply > Run`

**Result:** A JavaFX window appears with the greeting label.

### 11.1.2 2. macOS Setup

#### 2.1 Install Eclipse IDE

- Download and install Eclipse IDE for Java Developers from `https://www.eclipse.org/downloads/`
- Use the bundled JDK and Maven (no need to install Homebrew, JDK, or Maven separately)

#### 2.2 Download JavaFX SDK

1. Download JavaFX SDK (e.g. version 21) from `https://openjfx.io`
2. Extract it to:

```
/Users/yourname/JavaFX/javafx-sdk-21
```

#### 2.3 Create Maven Project

- Follow the same steps as in the Windows section to create a Maven project using the `maven-archetype-quickstart` archetype.

#### 2.4 Add JavaFX and MySQL Dependencies

- Open `pom.xml` and add the JavaFX and MySQL dependencies as shown in Section 1.4.

#### 2.5 Create Main JavaFX Class

- Same as Section 1.5 — create `MainApp.java` with JavaFX code.

#### 2.6 Run Project

- Same steps as in Section 1.6: create a Maven Run Configuration with:

```
clean javafx:run
```

**Note:** JavaFX SDK JARs are platform-specific. On macOS, Maven automatically downloads the correct `-mac` version from Maven Central.

—

### Conclusion

This setup allows full JavaFX GUI development using Maven in Eclipse across both Windows and macOS without external Maven or JDK installations. It serves as the basis for building more complex applications, such as the Cinema Manager app.